

שכבת הנתונים

שבוע: 11 ניקוד: 100 מועד הגשה: לפני תחילת שיעור 12 משקל בציר הסופי: מטלת כיתה 11 · 10 הטובות מתוך 12 נספרות

זמן משוער: כ-25 דקות עבודה בכיתה (145–175, אחרי חמש דקות הצגה; חלק א), והשלמה בבית

מטרה

להוסיף למזווה את שתי השכבות שמדברות עם העולם: `js/storage.js`, שבזכותו מה שנשמר נשאר מחר, ו- `js/api.js`, שמדבר עם קטלוג איטי שלפעמים לא עונה — ובלי ש- `render.js` ילמד על אף אחת מהן.

בשבוע 10 המזווה קיבל מקור אמת אחד. היום האמת הזאת שורדת רענון, ומבקשת משהו ממחשב של מישהו אחר. שתי שכבות חדשות, ו- `js/app.js` **לא משתנה**: הוא כבר קורא ל- `hydrate(load())` ורושם את `persist` — הם פשוט עוד לא עושים כלום. והשכבות האלה נכתבות **בחווה של הפרויקט**, מילה במילה. אחרי ההגשה אתה מעתיק את `js/storage.js` ואת `js/api.js` לפרויקט שלך, והבודק של הפרויקט והמבחן בשבוע 13 קוראים אותם בדיוק כמו הבודק של המטלה הזאת.

מה כבר הקלדת היום

המטלה היא **הרכבה** של חמשת המחזורים, על יישום אחר ובשמות אחרים — המזווה במקום ספריית ההשאלה, `pantry:v1` במקום `library:v1`, `searchTitles` במקום `searchCatalogue`. שום דבר לא מועתק; כל דבר מזווה.

חלק	מה מרכיבים	איפה הקלדת את זה
א	<code>load</code> · <code>save</code> · <code>persist</code> עם שני השומרים, המפתח <code><app>:v1</code> , מערך חשוף, ומפתח שאינו קיים = מזווה ריק	מחזור 1, ומחזור 2 צעד 3
ב	עריכה שנשמרת, <code>try</code> סביב הגישה וה- <code>parse</code> , <code>Array.isArray</code> , והשמטה של שורה פגומה	ה- <code>you-do</code> של מחזור 1, ומחזור 2
ג	<code>fetch</code> עם <code>res.ok</code> בשורה הבאה, פסק זמן, <code>res.json()</code> ב- <code>try</code> משלו, ארבע השאלות	מחזור 3 (וארבע השאלות — ה- <code>you-do</code> שלו)
ד	שדה אחד, חמישה ערכים, <code>data-state</code> , והנתיב הלא-מקוון	מחזור 4 צעדים 1–2, ומחזור 3 צעד 4
ה	כפתור שמוסיף הצעה לאוסף, והצעות שנגזרות	מחזור 4 צעד 3
ו	השהיה, ביטול, והמספר הסידורי · שומר הזהות	השהיה וביטול — ה- <code>you-do</code> של מחזור 4; המספר הסידורי לא הוקלד: הוא החידוש של המטלה, והבעיה שהוא פותר נלמדה במחזור 4 (שקופית 69 <code>w11-search</code>) <code>box-answers-wrong</code>) · שומר הזהות — מחזור 1 צעד 5
ז	ניסיון חוזר, "נקה הכול" עם אישור, והתקציב שנמדד	לא הוקלדו במחזור: ניסיון חוזר ו- <code>confirm</code> הם ההרחבה של המטלה; הרעיון — מתי שואלים שוב ומתי שואלים את המשתמש — נשען על ארבעת הכשלים של מחזור 4 (שקופית 65 <code>w11-four-failures-whateach-one</code>); התקציב — המונה של מחזור 1 צעד 5

בכיתה יש לך כ-25 דקות עבודה (145–175, אחרי חמש דקות שבהן המרצה מציג את המטלה ואת מטלת הבית 4): **חלק א** – שכבת האחסון, שמחזורים 1 ו-2 בנו לפני שעתיים על ספריית ההשאלה – וקומיט ראשון כשפריט שורד רענון. את השאר משלימים בבית, עד תחילת שיעור 12.

תוצרי למידה

- 1. לכתוב שכבת אחסון שבה **כל גישה** ל- `localStorage` עטופה, ושלעולם לא זורקת.
- 2. לדעת שמפתח שאינו קיים הוא **אוסף ריק**, ולהסביר למה זריעה של נתוני דוגמה ברגע כזה מסתירה תקלה.
- 3. להגן על היישום מפני מחרוזת פגומה, ומפני **JSON תקין לחלוטין שאינו הנתונים שלך**.
- 4. לדעת מה `fetch` **לא** עושה: הוא לא זורק על 404 או על 422, אין לו פסק זמן, והתשובה אינה הנתונים.
- 5. לחבר את **חמשת** מצבי הפאנל לשדה אחד, כך ששניים מהם לא יכולים להיות נכונים יחד.
- 6. לסגור מעגל: פריט שהגיע מהרשת נכנס לאוסף ושורד רענון – בלי שורת שמירה אחת.

מה יש בתיקייה

קובץ	מה עושים איתו
<code>js/storage.js</code>	הקובץ הראשון . שלד עם החוזה – המפתח כבר כתוב בו
<code>js/api.js</code>	הקובץ השני . שלד עם חוזה, בלי מימוש
<code>js/state.js</code>	שדה החיפוש ושתי גזירות
<code>js/events.js</code>	העריכה, "נקה הכול", ותזמור החיפוש
<code>js/render.js</code>	שורת ההצעה ואזור החיפוש. שום דבר על אחסון
<code>js/app.js</code>	נתון. ולא משתנה . כבר קורא ל- <code>hydrate(load())</code> ורושם את <code>persist</code>
<code>js/items.js</code>	נתוני הדוגמה. מגיעים רק מהכפתור "טען פריטים לדוגמה"
<code>data/catalogue.json</code>	תשובה שהוקלטה מה-API . ראה חלק ג
<code>summary.html · item.html · index.html</code>	נתונים ומלאים. לא נוגעים בהם

מריצים עם Live Server, תמיד. מ- file:// המודולים לא נטענים בכלל:

Access to script ... from origin 'null' has been blocked by CORS policy, ושום דבר בעמוד לא עובד.

החווה של האחסון הוא החווה של הפרויקט ושל המבחן: מפתח אחד בצורה <app>:v1 וקריאה אחת בלבד ל- getItem (דרישה 14 בברייף של הפרויקט) · הערך הוא **המעריך עצמו**, בלי מעטפה (דרישה 15 בודקת מערך) · **ומפתח שאינו קיים הוא מזווה ריק**, לא נתוני הדוגמה — את זה הברייף של הפרויקט לא כותב, והמבחן בשבוע 13 (גרסה ב) צריך אותו.

localStorage ו-JSON מופיעים ב- js/storage.js בלבד, ו- fetch וכתובת ה-API ב- js/api.js בלבד. לא ב- render.js, לא ב- events.js.

אף מטפל לא שומר בעצמו. subscribe(persist) ב- js/app.js הוא כל החיווט.

js/storage.js ו- js/api.js לא נוגעים ב-DOM, ו- js/api.js לא קורא ל- setState.

ב- js/render.js אין async ואין await.

localStorage.clear() אסור. הוא מוחק את כל ה-origin — גם את הפרויקטים האחרים שלך.

חלק א · שכבת האחסון · 16 נקודות ליבה

א.1 · load · החווה, ומפתח שאינו קיים

load() מחזירה תמיד { items, reason } · **הוא מעריך**, ו- reason הוא משפט בעברית כשמשוהו לא היה בסדר, או null. **היא לא זורקת לעולם** — היא רצה לפני הציור הראשון, וזריקה שם היא עמוד ריק שנראה בדיוק כמו render שבור.

מה קרה	מה מחזירים
getItem זרק	[] ומשפט — הדפדפן חוסם אחסון לאתר
getItem החזיר null	[] בלי משפט — ביקור ראשון, מזווה ריק
JSON.parse זרק	[] ומשפט — ואל תמחק את הערך : זה העותק היחיד
ה-JSON תקין ואינו מעריך	[] ומשפט
מעריך עם שורה פגומה אחת	השורות התקינות, ומשפט

השורה השנייה היא החווה. אתחול שזורע נתוני דוגמה כשהקריאה חוזרת ריקה — מה שחומרי הקורס עצמם לימדו עד השנה — לא יודע להבדיל בין "לא נשמר כלום" לבין "הקריאה נכשלה". בשבוע 13, בגרסה ב של המבחן, התקלה גורמת לקריאה לחזור ריקה: מזווה ריק הוא הסימפטום שתצטרך למצוא, וזריעה הייתה מסתירה אותו. נתוני הדוגמה מגיעים מהכפתור "טען פריטים לדוגמה" (נתון, ב- events.js).

א.2 · save ו- persist · שני השומרים

save(items) כותבת את המעריך, JSON.stringify(items), בתוך try, ומחזירה { ok, reason }. כתיבה שנכשלה **נאמרת**: { } catch ריק נותן יישום שנראה ששומר ואינו שומר.

`persist(state)` היא המנוי, ושני שומרים עוצרים אותה:

1. אין מה לכתוב כשהמערך הוא זה שנכתב בפעם הקודמת (משתנה ברמת המודול, השוואה לפי הפניה).
2. `setState` רק כשהמשפט השתנה. `persist` קוראת ל-`setState`, ו-`setState` מודיעה לכל המנויים — כולל לה. בלי התנאי הזה העמוד מת בלחיצה הראשונה: `Maximum call stack size exceeded`.

חלק ב · CRUD שנשמר, ונתונים שאינם נתונים · 11 נקודות ליבה

1. עריכה. `button.edit` כבר בכל שורה (`render.js` מצייר אותו). לחיצה ממלאת את הטופס (`fillForm`) ומסמנת את השורה (`editingId`); שליחה מחליפה את הפריט — `map`. עם `spread`, אותו מזהה, אותה כמות שורות — ויוצאת ממצב עריכה. זה אותו עדכון שהקלדת בעמוד הספר ב-you-do של מחזור 1.
2. עריכה ומחיקה שורדות רענון — ולא כתבת שורת שמירה.
3. זבל במפתח לא מפיל, וJSON תקין שאינו מערך לא מפיל. בסוף העבודה פתח את Application ב-DevTools וערוך את הערך ביד, ורענן אחרי כל אחד: `{}` · `null` · `{"not": "an array"}` · ומערך עם פריט אחד שלך ועוד `{"junk": true}` — שבו הפריט שלך נשאר.

חלק ג · שכבת הרשת · 15 נקודות ליבה

1.1 בחירת ה-API

בחר API מהרשימה שבסוף הבריק, או אחר — ובלבד שהוא עונה על ארבע שאלות: בלי מפתח? פתוח ל-CORS? `access-control-allow-origin` (בתשובה, בלשונית Network)? יודע על הישות? בקשה אחת לחיפוש? כתוב את ארבע התשובות בהערה בראש `js/api.js`, ועוד שלוש שורות על בקשה שה-CORS חסם (ה-you-do של מחזור 3: ה-API של ויקיפדיה בלי `origin=*`).

2.1 `searchTitles` ו-`searchOffline`

מה קרה	מה <code>searchTitles</code> עושה
הכול תקין	מחזירה מערך הצעות
לא נמצא כלום	מחזירה מערך ריק — זו לא שגיאה
404 · 422 · 500 · כל סטטוס	זורקת <code>ApiError</code> עם <code>kind: 'status'</code>
התשובה אינה JSON	זורקת <code>ApiError</code> עם <code>kind: 'shape'</code>
פסק הזמן פג	זורקת <code>ApiError</code> עם <code>kind: 'timeout'</code>
הרשת לא הגיעה	זורקת <code>ApiError</code> עם <code>kind: 'offline'</code>
הקורא ביטל	זורקת את ה- <code>AbortError</code> כמו שהוא

הכתובת נבנית עם `new URL` ו-`searchParams`. `res.ok` בשורה שאחריה `fetch` — `fetch` לא זורק על 404, על 422 ועל 500; אלה תשובות. `res.json()` ב-`try` משלו: 200 שהגוף שלו HTML עובר את `res.ok` ונופל שם.

`searchOffline` — אותו חיפוש, מול `data/catalogue.json`, **דרך אותו מנרמל**. הקובץ הוא **תשובה מוקלטת** מה-API שלך, במבנה שלו — לא רשימה שנכתבה ביד. פתח את הכתובת בדפדפן ושמור את מה שחזר, כמו שהוא.

מה ה-API שלך מחזיר כשלא נמצא כלום?

תנסה בשורת הכתובת לפני שתכתוב שורה. הרבה APIs משמיטים את מפתח התוצאות במקום להחזיר רשימה ריקה — ואז `payload.results.map` זורק `TypeError` מתוך `try` שנכתב בשביל הרשת.

חלק ד · חמשת המצבים והנתיב הלא-מקוון · 15 נקודות ליבה

`#api-panel` נושא `data-state`, ותמיד **בדיק אחת** מהמילים:

המצב	מתי
<code>idle</code>	עוד לא חיפשו — לא "לא נמצא כלום"
<code>loading</code>	בקשה פתוחה — נכתב לפני ה- <code>await</code>
<code>error</code>	בקשה נכשלה: משפט, ודרך קדימה
<code>empty</code>	בקשה הצליחה ולא מצאה — נגזר מהתוצאות
<code>results</code>	בקשה הצליחה ומצאה

שדה אחד עם ערכים, לא בוליאנים: `isLoading` לצד `isEmpty` הם "מחפש..." ו"לא נמצא" על המסך יחד — מחזור 4, צעד 1. `render` מחשב את התצוגה **פעם אחת** ומציג אזור אחד.

הנתיב הלא-מקוון: כשהבקשה נכשלה, מצב השגיאה **מציב** את הקטלוג המקומי (`#api-offline`) — מציע, לא לוקח — ו-`#api-source` אומר איזה קטלוג ענה.

חלק ה · מהקטלוג אל האוסף · 8 נקודות ליבה

- לכל הצעה `button.adopt`, ולשורה `data-title`. לחיצה מוסיפה פריט **במבנה של היישום שלך**.
- מטפל אחד** על `#api-results`; הנתונים מ-`state.search.results`, לא מהמסך.
- הצעה **שכבר באוסף אינה מוצעת שוב** — וזה **נגזר**: הסרת הפריט מחזירה את ההצעה.

הרגע שבו שתי השכבות נפגשות

המטפל הזה הוא חמש שורות, הוא לא שמע מעולם על `localStorage`, ומה שהוא מוסיף נמצא על הדיסק אחרי רענון. תוסיף פריט מהקטלוג, תרענן, ותסתכל.

חלק ו · המרוץ ותקציב הכתיבות · 20 נקודות הרחבה

- השהיה של ~300ms**: מילה בת שש אותיות היא בקשה אחת. תיבה ריקה חוזרת ל-`idle` מיד.
- ביטול**: חיפוש חדש מבטל את הקודם (`AbortController` אחד, ו-`AbortSignal.any` עם פסק הזמן).

3. **מספר סידורי:** אחרי כל `await`, לפני שכותבים ל-`state`, בודקים שהכרטיס עדיין הנוכחי.

4. **AbortError אינו שגיאה:** `return`, בלי לגעת בכלום.

5. **הקלדה בתיבת הסינון של הרשימה לא כותבת לדיסק** – השומר הראשון של `persist` (או השהיה עם `flush` ב-`pagehide`), ושינוי אמיתי כן נכתב.

חלק ז · ניסיון חוזר, ניקוי הכול, והתקציב שנמדד · 10 נקודות אתגר

1. **ניסיון חוזר** על חיבור שנפל או פסק זמן – לא על סטטוס, ובחזקה לא על `AbortError` – עם השהיה שגדלה וניתנת לביטול.
2. `#clear-all` **שואל קודם** (`confirm`), ומה שנוקה נשאר ריק אחרי רענון.
3. **מדוד:** כמה כתיבות עלתה הקלדת מילה בתיבת הסינון בלי השומר הראשון, וכמה איתו – שני המספרים בהערה ב-`js/storage.js`. אם בחרת השהיה – `flush` ב-`pagehide`.

כללי · 5 נקודות ליבה

מה	כמה
אין <code>var · innerHTML</code> מקבל רק מחרוזת קבועה מהקוד (למשל <code>' '</code> לניקוי) – לא משתנה, לא קריאה לפונקציה ולא תבנית עם <code>{...}</code>	2
היגיינת קומיטים: קומיט לכל חלק, הודעה שאומרת מה השתנה	3

רשימת ה-APIs

נבדקו ב-19 באוגוסט 2026: בלי מפתח, פתוחים ל-CORS, מחזירים JSON. ובכל זאת – **בדוק בעצמך** לפני שאתה בונה (ג.1): הכותרת היא החלטה של השרת, והיא משתנה.

ה-API	כתובת	טוב בשביל
ויקיפדיה העברית	<code>he.wikipedia.org/w/api.php</code> (עם <code>origin=*</code>)	כל ישות שיש עליה ערך. בעברית
Open Library	<code>openlibrary.org/search.json</code>	ספרים, מחברים (שאלה של פחות משלוש אותיות < 422)
iTunes Search	<code>itunes.apple.com/search</code>	מוזיקה, סרטים, פודקאסטים
TheMealDB	<code>themaaldb.com/api/json/v1/1/</code>	מתכונים ומרכיבים
Open-Meteo	<code>api.open-meteo.com/v1/forecast</code>	מזג אוויר
REST Countries	<code>restcountries.com/v3.1</code>	מדינות, ערי בירה
Frankfurter	<code>api.frankfurter.dev/v1/latest</code>	שערי מטבע
PokéAPI	<code>pokeapi.co/api/v2</code>	אם הישות שלך מתאימה
Dog CEO	<code>dog.ceo/api</code>	תמונות, גזעים

API-ה	כתובת	טוב בשביל
Datamuse	<code>api.datamuse.com/words</code>	מילים באנגלית
GitHub REST	<code>api.github.com</code>	מאגרים ומשתמשים. 60 בקשות לשעה

ובקשה אחת שנחסמת, בכוונה

he.wikipedia.org/w/api.php?...&format=json בלי *origin= — API של ויקיפדיה שולח את הכותרת רק למי שמבקש אותה, והדפדפן חוסם: `TypeError: Failed to fetch`, ובקונסול "blocked by CORS policy". עם *origin= — עובד. (נבדק ב-27 בספטמבר 2026, data.gov.il, שהחומרים של שנה שעברה הציגו כחסום, החזיר באותו יום את הכותרת — עוד סיבה לבדוק ולא לסמוך על רשימה.)

מדיניות AI לשבוע הזה

שותף למימוש (שבועות 8–12). Claude Code על התיקיה, או הצ'אט עם הקבצים מודבקים: אותו כלל, אותה סקירה, אותו ציון. שכבות אחסון ורשת הן בדיקת הקוד ש-Claude כותב הכי מהר והכי משכנע — וזו הבעיה: הגרסה שחוזרת בברירת מחדל עובדת עד הפעם הראשונה שמשו לא צפוי. שלוש טעויות שראית במחזור 5, ושלושתן עוברות כל בדיקה ידנית ברשת טובה:

- `persist` ו-`setState` שקוראות זו לזו — העמוד לא עולה.
 - `JSON.parse` בלי `try` ובלי בדיקת מבנה — `{}` במפתח מפיל אותו.
 - `fetch` בלי `res.ok` — 422 מוצג כ"לא נמצא כלום".
- ועוד שתיים מהחודה: מפתח בשם `items` או מעטפה סביב המערך, ו-`SEED_ITEMS ?? load()`. **כתוב את הכללים בפרומפט, וקרא את הפלט מול הטבלאות של חלקים א ו-ג לפני שאתה מריץ.**
- בשכבת השותף, מסלול הצ'אט: הדבק את הקבצים לצ'אט, העתק את מה שחוזר ידנית, ובצע את אותה סקירה — הציון זהה. אין לך מנוי? הכול עובד גם בצ'אט החינמי, ואין בכך שום חיסרון בקורס.

הגשה

תיקיה week-11/ במאגר הסמסטר שלך.

הגשה במודל: כתובת המאגר + ה-SHA של הקומיט. ה-SHA מקפיא את מה שנבדק.
אחרי ההגשה, העתק את `js/storage.js` ואת `js/api.js` ל-`project/` שלך — הם כתובים בחוזה שלו. **הפרויקט מוגש בשבוע 12 עם נתונים שנשמרים ונתונים חיים**, וזה השבוע שבו זה נעשה.
הגשה מאוחרת: הגשה באיחור אינה מותרת ללא אישור מראש בכתב. הגשה באיחור תגרור הפחתה בציון.

סיימת כשכל אלה נכונים

- ☐ טענת את הדוגמאות, הוספת פריט, רעננת — הכול שם.
- ☐ מחקת את המפתח ב-Application ורעננת — **מזווה ריק**, עם המשפט והכפתור. לא נתוני הדוגמה.
- ☐

- . הערך ב-Application מתחיל ב- [] – המערך עצמו, והמפתח נגמר ב- v1: .
- . ערכת פריט ורעננת – הערך החדש שם, ואותה כמות שורות.
- . {{{ · null · {"not": "an array"} במפתח – העמוד עולה בכל אחד, עם משפט.
- . הקלדת מילה בקטלוג: תוצאות, ו- data-state="results" ב-Elements.
- . כיבית את ה-Wi-Fi והקלדת: משפט, והקטלוג המקומי עובד.
- . לחצת על הצעה, רעננת – הפריט עדיין שם, וההצעה לא מוצעת שוב.
- . המילה localStorage לא מופיעה ב-render.js וב-events.js, והמילה fetch לא ב-render.js.
- . js/app.js לא השתנה. תריץ git diff עליו ותוודא.